



TECHNICAL TRAINING & PROJECT REPORT

PYTHON PROGRAMMING AND PROJECT DEVELOPMENT

Submitted By

Ashwina Pandey

Roll No.: 25SCS1003004835

B.Tech – Computer Science & Engineering

Semester: 3rd Semester

Section: 2CSE4

Session: 2025–29

Technical Training Course

100 Days of Code™: The Complete Python Pro Bootcamp

Instructor: Dr. Angela Yu

Platform: Udemy

DECLARATION

I hereby declare that the work presented in this report is based on my learning and practical implementation during the completion of the **100 Days of Code™: The Complete Python Pro Bootcamp**.

This report presents the Python programming concepts studied during the training and the practical projects developed to apply those concepts. The projects included in this report were developed as part of my technical learning and are presented for academic evaluation.

Name: Ashwina Pandey

Roll No.: 25SCS1003004835

Section: 2CSE4

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my college and faculty members for providing me with the opportunity to enhance my technical knowledge through self-paced learning and practical project development.

I am thankful to **Dr. Angela Yu**, instructor of the *100 Days of Code™: The Complete Python Pro Bootcamp*, for providing structured learning material and practical exercises that helped me develop a strong foundation in Python programming.

The practical and project-based nature of the course allowed me to apply programming concepts while developing different applications. This experience helped me improve my logical thinking, programming skills, debugging abilities and confidence in developing Python programs.

I would also like to thank everyone who supported and encouraged me throughout the learning and project development process.

TABLE OF CONTENTS

1. Introduction
2. Training Objectives
3. Course Overview
4. Topics and Concepts Learned
5. Development Environment and Tools
6. Projects Developed
7. Password Generator
8. Hangman Game
9. Rock Paper Scissors
10. Caesar Cipher

11. Blind Auction
 12. Skills Acquired
 13. Challenges and Solutions
 14. Learning Outcomes
 15. Future Scope
 16. Conclusion
 17. References
-

1. INTRODUCTION

Python is a high-level, interpreted programming language known for its simple syntax, readability and versatility. It is widely used in areas such as software development, automation, data science, artificial intelligence, web development and application development.

As part of my technical training, I completed the **100 Days of Code™: The Complete Python Pro Bootcamp**, instructed by **Dr. Angela Yu** on Udemy. The course followed a practical and project-oriented approach to learning Python programming.

During the training, I studied fundamental programming concepts and applied them through practical projects. The projects helped me understand how programming concepts can be combined to develop functional applications.

As part of my practical learning, I developed five Python projects:

1. **Password Generator**
2. **Hangman Game**
3. **Rock Paper Scissors**
4. **Caesar Cipher**
5. **Blind Auction**

These projects provided practical experience with concepts such as variables, conditional statements, loops, functions, strings, lists, dictionaries, randomization and user input.

The purpose of this report is to document the technical training, concepts learned, projects developed and skills acquired during the learning process.

2. TRAINING OBJECTIVES

The main objectives of the technical training were:

- To understand the fundamentals of Python programming.
 - To develop logical and computational thinking.
 - To understand variables, data types and operators.
 - To implement conditional statements and loops.
 - To understand and use functions.
 - To work with Python data structures.
 - To understand string manipulation.
 - To implement randomization in Python applications.
 - To develop interactive console-based applications.
 - To improve debugging and problem-solving skills.
 - To apply theoretical programming concepts through practical projects.
 - To gain confidence in developing small-scale Python applications.
-

3. COURSE OVERVIEW

3.1 Course Information

Particular	Details
Course Name	100 Days of Code™: The Complete Python Pro Bootcamp
Instructor	Dr. Angela Yu
Platform	Udemy
Domain	Python Programming

Learning Mode	Self-Paced
Status	Completed
Practical Component	Python Projects

3.2 About the Course

The **100 Days of Code™: The Complete Python Pro Bootcamp** is a project-oriented Python programming course designed to develop programming skills through structured learning and practical implementation.

The course introduces Python programming concepts and gradually develops the learner's ability to apply those concepts to practical problems. The project-based learning approach provides opportunities to implement programming logic and develop functional applications.

The training helped me build a foundation in Python and understand how programming concepts can be applied to solve different problems.

4. TOPICS AND CONCEPTS LEARNED

During the training, I worked with several important Python programming concepts.

4.1 Variables and Data Types

Variables are used to store information during program execution. I learned to work with different Python data types such as integers, floating-point numbers, strings and Boolean values.

4.2 Operators

I learned to use arithmetic, comparison and logical operators to perform calculations and construct conditions within programs.

4.3 Conditional Statements

Conditional statements allow programs to make decisions based on specific conditions. I worked with:

- `if`
- `elif`
- `else`

These concepts were particularly useful in game development and determining different program outcomes.

4.4 Loops

Loops allow instructions to be executed repeatedly. I worked with:

- `for` loops
- `while` loops

Loops were useful for repeated user interaction, gameplay and processing collections of information.

4.5 Functions

Functions allow a program to be divided into reusable blocks of code. They improve code organization, readability and reusability.

4.6 Strings

String manipulation was an important part of the training and was particularly useful in the **Caesar Cipher** project.

4.7 Lists

Lists allow multiple values to be stored and processed together. They were useful for managing collections of information within different programs.

4.8 Dictionaries

Dictionaries store information in key-value pairs. This concept was particularly useful in the **Blind Auction** project for storing bidder names and their bids.

4.9 Randomization

Randomization was used in projects where unpredictable selections were required, such as games and password generation.

5. DEVELOPMENT ENVIRONMENT AND TOOLS

5.1 Python

Python was the primary programming language used to develop the projects.

5.2 PyCharm

PyCharm was used as the Integrated Development Environment (IDE) for writing, running and debugging Python programs.

It provided useful features such as:

- Code editing
- Syntax highlighting
- Error detection
- Program execution
- Debugging

5.3 GitHub

GitHub is used to organize and store the completed project files and supporting documentation. It also provides a convenient way to maintain and share project source code.

6. PROJECTS DEVELOPED

As part of the practical component of the training, I developed five console-based Python applications.

The projects were:

6.1 Password Generator

A Python application that generates random passwords based on user requirements.

6.2 Hangman Game

An interactive word-guessing game where the player attempts to identify a hidden word.

6.3 Rock Paper Scissors

A game where the player competes against the computer using the traditional Rock Paper Scissors rules.

6.4 Caesar Cipher

A text encryption and decryption application based on shifting characters by a specified number of positions.

6.5 Blind Auction

A console-based auction application that accepts bids from multiple participants and identifies the highest bidder.

These projects allowed me to apply Python concepts in different practical scenarios.

7. PROJECT 1 — PASSWORD GENERATOR

7.1 Project Overview

The Password Generator is a Python-based console application designed to generate random passwords. The project demonstrates how Python can be used to combine different characters and generate passwords automatically.

7.2 Objective

The main objectives of this project were:

- To generate random passwords.
- To practice string and list manipulation.
- To work with randomization.
- To apply loops and programming logic.
- To create an interactive console application.

7.3 Key Features

- Generates passwords automatically.
- Uses different types of characters.
- Accepts user requirements.
- Produces the generated password through the console.

7.4 Python Concepts Used

- Variables
- Strings
- Lists
- Loops
- User input
- Randomization
- Functions

7.5 Working

The program accepts the required password parameters from the user and uses Python programming logic to generate a password. Characters are selected and arranged to produce the final password.

7.6 Learning Outcome

This project helped me understand randomization, string manipulation, lists and user input while developing a practical Python application.

```
1 import random
2 letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r',
3           's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
4
5 numbers = ['1', '2', '3', '4', '5', '6', '7', '8', '9', '0']
6
7 symbols = ['!', '#', '$', '%', '^', '&', '*', '(', ')', '+']
8
9 print("welcome to the pypassword generator")
10
11 nr_letter = int(input("enter the number of letters: "))
12 nr_symbol = int(input("enter the number of symbols: "))
13 nr_number = int(input("enter the number of numbers: "))
14
15 password_list = []
16
17 for char in range(0, nr_letter):
18     password_list.append(random.choice(letters))
19 for char in range(0, nr_symbol):
20     password_list.append(random.choice(symbols))
21 for char in range(0, nr_number):
22     password_list.append(random.choice(numbers))
23
24 print(password_list)
25 random.shuffle(password_list)
26 print(password_list)
27
28 password = ""
29 for char in password_list:
30     password += char
31 print(f"your password is: {password}")
```

```
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Documents/coding$ cd python
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Documents/coding/python$ python password.py
welcome to the pypassword generator
enter the number of letters: 4
enter the number of symbols: 5
enter the number of numbers: 6
['r', 'x', 'u', 'v', '#', '+', '#', '#', '%', '3', '6', '4', '4', '9', '7']
['4', '9', '+', '4', 'v', '6', 'x', '#', 'r', '%', 'u', '3', '#', '#', '7']
your password is: 49+4v6x#r%u3##7
```

8. PROJECT 2 — HANGMAN GAME

8.1 Project Overview

Hangman is an interactive word-guessing game developed using Python. The player attempts to identify a hidden word by guessing individual letters.

8.2 Objective

The objectives of this project were:

- To develop an interactive game using Python.
- To practice loops and conditional statements.
- To work with strings and lists.
- To handle repeated user input.
- To implement game logic.

8.3 Key Features

- Interactive word guessing.
- Letter-based input.
- Limited attempts.
- Displays the progress of the guessed word.
- Determines whether the player wins or loses.

8.4 Python Concepts Used

- Strings
- Lists
- Loops
- Conditional statements
- Functions
- Randomization

8.5 Working

The program selects a word and hides its characters from the player. The player enters guesses, and the program checks whether the guessed letter exists in the selected word. The game continues until the player successfully identifies the word or reaches the maximum number of incorrect attempts.

8.6 Learning Outcome

This project improved my understanding of loops, conditional statements, strings, lists and interactive program logic.

```
75 word_list = ["aardvark", "baboon", "camel"]
76
77 lives = 6
78
79 chosen_word = random.choice(word_list)
80 print(chosen_word)
81 placeholder = ""
82 word_length = len(chosen_word)
83 for position in range(word_length):
84     placeholder += "_"
85 print(placeholder)
86
87 game_over = False
88 correct_letters = []
89
90 while not game_over:
91     guess = input("Guess a letter: ")
92     display = ""
93     for letter in chosen_word:
94         if letter == guess:
95             display += letter
96             correct_letters.append(letter)
97         elif letter in correct_letters:
98             display += letter
99         else:
100             display += "_"
101     print(display)
102     if guess not in chosen_word:
103         lives -= 1
104         if lives == 0:
105             game_over = True
106             print("You Lose!")
```

```
if "_" not in display:
    game_over = True
    print("You win!")

print(stages[lives])
```

```
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Documents/coding/python$ python hangman.py
baboon
```

```
Guess a letter: h
```

```

+---+
|   |
O   |
    |
    |
=====
```

```
Guess a letter: j
```

```

+---+
|   |
O   |
|   |
    |
    |
=====
```

```
Guess a letter: p
```

```

+---+
|   |
O   |
/|   |
    |
    |
=====
```

```
Guess a letter: k
```

```

+---+
|   |
O   |
/|\  |
    |
    |
=====
```

```
Guess a letter: b
b _ _ _
```

```

+---+
|   |
O   |
/|\  |
    |
    |
=====
```

```
Guess a letter: g
b _ _ _
```

```

+---+
|   |
O   |
/|\  |
/    |
    |
=====
```

```
Guess a letter: j
b _ _ _
You Lose!
```

```

+---+
|   |
O   |
/|\  |
    |
    |
=====
```

9. PROJECT 3 — ROCK PAPER SCISSORS

9.1 Project Overview

Rock Paper Scissors is a console-based game in which the player competes against the computer. The winner is determined according to the standard rules of the game.

9.2 Objective

The objectives were:

- To implement game logic using Python.
- To use randomization.
- To process user input.
- To implement conditional decision-making.

9.3 Key Features

- Player selection.
- Computer selection.
- Random computer choice.
- Automatic winner determination.
- Interactive gameplay.

9.4 Python Concepts Used

- Conditional statements
- User input
- Randomization
- Loops
- Functions
- Variables

9.5 Working

The player selects Rock, Paper or Scissors. The computer makes its selection using randomization. The program then compares both selections and determines the winner based on the rules of the game.

9.6 Learning Outcome

This project helped me understand conditional logic, randomization and user interaction while developing an interactive program.

```
2 import random
3
4 rock = """
5
6 ---'-----)
7 |             )
8 |             )
9 |             )
10 ---'-----)
11 """
12
13 paper = """
14
15 ---'-----)
16 |             )
17 |             )
18 |             )
19 ---'-----)
20 """
21
22 scissors = """
23
24 ---'-----)
25 |             )
26 |             )
27 |             )
28 ---'-----)
29 """
```

```
30 game_images = [rock, paper, scissors]
31 user_choice = int(input("what do you choose, type 0 for rock, type 1 for paper, type 2 for scissors: "))
32 if user_choice >= 0 and user_choice <= 2:
33     print(game_images[user_choice])
34     computer_choice = random.randint(0,2)
35     print("computer chose:")
36     print(game_images[computer_choice])
37     if user_choice >= 3 or user_choice < 0:
38         print("you chose an invalid number")
39     elif computer_choice == 0 and user_choice == 2:
40         print("you lose")
41     elif user_choice == 0 and computer_choice == 2:
42         print("you win")
43     elif user_choice > computer_choice:
44         print("you win")
45     elif computer_choice > user_choice:
46         print("you lose")
47     elif computer_choice == user_choice:
48         print("its a draw")
```

```
aashi@Pawan-Laptop-Personal: /mnt/c/Users/Dell/OneDrive/Documents/coding/python$ python rockpaperscissor
what do you choose, type 0 for rock, type 1 for paper, type 2 for scissors: 2
```

```
---'-----)
|             )
|             )
|             )
---'-----)
```

computer chose:

```
---'-----)
|             )
|             )
|             )
---'-----)
```

you win

10. PROJECT 4 — CAESAR CIPHER

10.1 Project Overview

Caesar Cipher is a basic text encryption and decryption application. It uses character shifting to transform the original message into an encoded message.

10.2 Objective

The objectives of this project were:

- To understand basic encryption concepts.
- To practice string manipulation.
- To work with character positions.
- To implement encryption and decryption logic.
- To develop problem-solving skills.

10.3 Key Features

- Text encryption.
- Text decryption.
- User-defined shift.
- Interactive input and output.
- Character-based transformation.

10.4 Python Concepts Used

- Strings
- Lists
- Loops
- Conditional statements
- Functions
- Character manipulation

10.5 Working

The program takes a message and a shift value as input. Each character is shifted by the specified number of positions to generate the encrypted message. The reverse process can be used to decode the message.

For example, with a shift of 3:

Original: HELLO

Encrypted: KHOOR

10.6 Learning Outcome

This project improved my understanding of strings, loops, functions and character manipulation while introducing the basic concept of text encryption.

```
9 alphabet = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l',  
10            'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']  
11  
12  
13  
14 def caesar(original_text, shift_amount, encode_or_decode):  
15     output_text = ""  
16     if encode_or_decode == "decode":  
17         shift_amount *= -1  
18     for letter in original_text:  
19         if letter not in alphabet:  
20             output_text += letter  
21         else:  
22             shifted_position = alphabet.index(letter) + shift_amount  
23             shifted_position %= len(alphabet)  
24             output_text += alphabet[shifted_position]  
25     print(f"here is the {encode_or_decode}d text: {output_text}")  
26  
27  
28 should_continue = True  
29 while should_continue:  
30     direction = input("type 'encode' to encode text or 'decode' to decode text: ")  
31     text = input("type your text: ").lower()  
32     shift = int(input("type the shift number: "))  
33  
34     caesar(original_text=text, shift_amount=shift, encode_or_decode=direction)  
35     restart = input("type 'yes' to continue or 'no' to exit:\n").lower()  
36  
37 if restart == "no":  
38     should_continue = False  
39     print("Thank you for using caesar!")
```

```
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Documents/coding/python$ python caesar.py  
  
,adPPYba, ,adPPYba, ,adPPYba, ,adPPYba, ,adPPYba,  
a8" "" `Y8 a8P__88 a8P__88 a8P__88  
8b ,adPPPP88 8pp"""""" 8pp"""""" 8pp""""""  
"8a, ,aa 88, ,88 "8b, ,aa "8b, ,aa  
`"Ybbd8" `8bbdP"Y8 `Ybbd8" `Ybbd8" `Ybbd8"  
  
type 'encode' to encode text or 'decode' to decode text: encode  
type your text: hello  
type the shift number: 4  
here is the encoded text: lipps  
type 'yes' to continue or 'no' to exit:
```

11. PROJECT 5 — BLIND AUCTION

11.1 Project Overview

Blind Auction is a console-based Python application that simulates an auction. Participants enter their names and bids, after which the program determines the participant with the highest bid.

11.2 Objective

The objectives were:

- To simulate an auction process.
- To store participant information.
- To work with dictionaries.
- To compare numerical values.
- To determine the highest bidder.

11.3 Key Features

- Supports multiple bidders.
- Stores bidder information.
- Accepts bid values.
- Compares bids.
- Identifies the highest bidder.

11.4 Python Concepts Used

- Dictionaries
- Loops
- Conditional statements
- Functions
- User input
- Comparison operations

11.5 Working

The program accepts the name and bid amount of each participant. The information is stored using a dictionary. After all participants have entered their bids, the program compares the values and determines the participant with the highest bid.

11.6 Learning Outcome

This project helped me understand dictionaries, loops, conditional logic and data comparison while developing a practical simulation.

```
1 logo = r'''
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
'''
print(logo)

def find_highest_bidder(bidding_dictionary):
    winner = ""
    highest_bid = 0
    for bidder in bidding_dictionary:
        bid_amount = bidding_dictionary[bidder]
        if bid_amount > highest_bid:
            highest_bid = bid_amount
            winner = bidder
    print(f"the winner is {winner} with the highest bid of {highest_bid}")
```

```
bids = {}
continue_bidding = True
while continue_bidding:
    name = input("Enter your name: ")
    price = int(input("what is your bid? $"))
    bids[name] = price
    should_continue = input("are there any other bidders? type 'yes' or 'no':\n ").lower()
    if should_continue == "no":
        continue_bidding = False
        find_highest_bidder(bids)
    elif should_continue == "yes":
        print("\n"*20)
```

```
aashi@Pawan-Laptop-Personal: /mnt/c/Users/Dell/OneDrive/Documents/coding/python$ python blindauction.py
```

```

Enter your name: ashwina
what is your bid? $200
are there any other bidders? type 'yes' or 'no':
yes
```

```
Enter your name: ananya
what is your bid? $300
are there any other bidders? type 'yes' or 'no':
no
the winner is ananya with the highest bid of 300
```

12. PROJECT COMPARISON

The five projects provided practical exposure to different Python programming concepts.

Project	Main Concepts Applied
Password Generator	Strings, Lists, Randomization, Loops
Hangman Game	Strings, Lists, Loops, Conditions
Rock Paper Scissors	Conditions, Randomization, User Input
Caesar Cipher	Strings, Loops, Functions, Character Manipulation
Blind Auction	Dictionaries, Loops, Conditions, Comparisons

The projects demonstrated how the same programming language can be used to create different types of applications, including games, utilities, text-processing programs and simulations.

13. SKILLS ACQUIRED

13.1 Technical Skills

Through the training and project development, I developed the following technical skills:

- Python programming
- Problem-solving
- Logical thinking
- Programming fundamentals
- String manipulation
- Data structure usage
- Randomization
- Function-based programming
- Console application development
- Basic debugging

13.2 General Skills

The training also helped me develop:

- Independent learning
 - Analytical thinking
 - Debugging and troubleshooting
 - Time management
 - Attention to detail
 - Project organization
 - Technical documentation
-

14. CHALLENGES AND SOLUTIONS

During project development, I encountered several challenges while implementing programming logic and handling user interaction.

Challenge 1 — Implementing Program Logic

Developing correct conditions for games required careful consideration of different possible inputs and outcomes.

Solution: I divided the problem into smaller logical conditions and tested different scenarios.

Challenge 2 — Handling User Input

User input can vary and may not always match the expected format.

Solution: I used appropriate conditions and program logic to handle different inputs and provide suitable responses.

Challenge 3 — Debugging

Errors in syntax and program logic occurred during development.

Solution: I used the debugging and error-detection features of PyCharm and tested the programs repeatedly to identify and correct errors.

Challenge 4 — Managing Data

Projects such as Blind Auction required information from multiple participants to be stored and processed.

Solution: Python data structures such as dictionaries were used to organize and process the information.

15. LEARNING OUTCOMES

After completing the technical training and developing the projects, I gained practical experience in writing Python programs and applying programming concepts to different problems.

I developed a better understanding of:

- Python syntax and programming structures.
- Conditional statements and loops.
- Functions and reusable code.
- Strings and data structures.
- Randomization.
- User interaction.
- Program testing and debugging.

The project-based learning approach also improved my ability to break a problem into smaller components and develop a logical solution.

16. FUTURE SCOPE

The projects developed during the training can be further enhanced by introducing additional functionality.

The console-based applications can be converted into graphical applications using Python GUI libraries. The projects can also be improved by adding file storage, databases, better input validation and graphical user interfaces.

For example, the Password Generator can be extended with password-strength analysis, while the games can include scoring systems, multiple difficulty levels and improved user interfaces.

The Caesar Cipher can be expanded into a larger text-encryption application, and the Blind Auction can be enhanced using persistent data storage and a graphical interface.

These improvements would provide opportunities to apply more advanced programming concepts in future projects.

17. CONCLUSION

The **100 Days of Code™: The Complete Python Pro Bootcamp** provided a valuable opportunity to develop programming knowledge through structured learning and practical implementation.

The development of the **Password Generator, Hangman Game, Rock Paper Scissors, Caesar Cipher and Blind Auction** allowed me to apply different Python concepts in practical

scenarios. These projects strengthened my understanding of programming logic, conditional statements, loops, functions, strings, data structures, randomization and user interaction.

Overall, the training helped me establish a strong foundation in Python programming and improved my confidence in developing small-scale applications. The practical experience gained through these projects will also provide a foundation for learning more advanced programming concepts and technologies in the future.

18. REFERENCES

1. *100 Days of Code™: The Complete Python Pro Bootcamp* — Dr. Angela Yu, Udemy.
 2. Python programming documentation and learning resources used during the training.
 3. PyCharm Integrated Development Environment and its documentation.
 4. Additional programming references consulted during project development.
-

ANNEXURE

A. Project Screenshots

The following screenshots provide evidence of the practical implementation and execution of the projects:

- Password Generator — Source Code and Output
- Hangman Game — Source Code and Output
- Rock Paper Scissors — Source Code and Output
- Caesar Cipher — Source Code and Output
- Blind Auction — Source Code and Output

B. Course Completion Certificate



Certificate no: UC-ccd9b34-751a-4be4-a0bf-2084fbb9a283
Certificate url: ude.my/UC-ccd9b34-751a-4be4-a0bf-2084fbb9a283
Reference Number: 0004

CERTIFICATE OF COMPLETION

100 Days of Code™: The Complete Python Pro Bootcamp

Instructors **Dr. Angela Yu, Developer and Lead Instructor**

Ashwina

Date **Aug. 29, 2026**

Length **57 total hours**
